

RF-005: Gestão de Usuários e Consolidação (MVP v1.0.0)

1. IDENTIFICAÇÃO DO REQUISITO

ID: RF-005

Título: Consolidação MVP v1.0.0 (Gestão de Pessoas, Senhas e Acessos)

Tipo: Requisito Funcional e de Segurança

Prioridade: ALTÍSSIMA (Consolida a entrega final do Produto Mínimo Viável)

Complexidade: MÉDIA (Envolve UI Condicional e Segurança Stateful)

Status: CONCLUÍDO (MVP v1.0.0)

Data de Criação: 23/09/2026

Última Atualização: 28/09/2026

Breve Descrição: Implementação do módulo de gestão administrativa de funcionários e clientes (CRUD com deleção lógica), estabelecendo o motor de hierarquia de privilégios. Inclui a criação de um painel de autoatendimento ("Meu Painel") para troca segura de credenciais e atualização de dados próprios, cancelando a versão base (v1.0.0) do ERP de acordo com os critérios de segurança OWASP e restrições de arquitetura exigidos pela avaliação acadêmica.

2. DESCRIÇÃO E ATORES

Contexto do negócio: O ERP atingiu sua maturidade operacional. Agora, a loja precisa gerenciar quem acessa o sistema. Vendedores lidam apenas com clientes (balcão), enquanto Gestores e Administradores administram a folha de acessos da equipe. Nenhuma senha pode ser criada pelo front-end para novos funcionários, e a inativação de um perfil deve revogar imediatamente seu acesso sem destruir o histórico de vendas.

Atores do Sistema:

1. Administrador e Gestor (Atores Principais)

- **Papel:** Gerenciar a equipe e a carteira completa de clientes.
- **Permissões:**
 - CREATE (Novos funcionários e clientes).
 - READ (Visão global de todos os usuários e clientes, ativos ou inativos).
 - UPDATE (Editar dados e inativar perfis).

2. Vendedor (Ator Secundário Restrito)

- **Papel:** Operador de balcão.
- **Permissões:**
 - READ / UPDATE / DELETE lógico (Restrito **apenas** a Clientes onde **user_id IS NULL**).
 - Cego para a existência de outros funcionários no sistema.

3. Sistema (Ator Automático)

- **Papel:** Leão de chácara da segurança e gerador de hashes.

- **Permissões:**

- Gerar a credencial inicial `senha123` via `bcrypt` para novos cadastros.
- Barrar acessos de tokens válidos cujo perfil esteja marcado como `ativo = false`.

3. ESPECIFICAÇÃO DE CASOS DE USO E REQUISITOS NÃO-FUNCIONAIS

Caso de Uso (UC-005): Cadastro de Funcionário e Gestão de Credenciais

Pré-Condições

- Usuário logado com sessão Stateful ativa (`token_uuid` validado no banco).
- Tabelas `users` e `pessoas` atualizadas com a coluna `ativo BOOLEAN NOT NULL DEFAULT TRUE`.

Pós-Condições (Sucesso)

- Novo funcionário criado simultaneamente nas tabelas `users` (credencial) e `pessoas` (perfil).
- Senhas alteradas com sucesso a partir do ID extraído estritamente do Token.

Pós-Condições (Falha)

- Vendedor tentando listar funcionários, ou Gestor tentando criar Admin, recebem erro `403 Forbidden`.

Fluxo Principal (Criação de Funcionário por Gestor)

1. Gestor acessa a página "Gestão de Pessoas" (`gestaoPessoas.html`).
2. Frontend valida o RBAC Client-Side e exibe o botão "+ Novo Funcionário".
3. Gestor clica no botão, preenchendo Nome, E-mail e selecionando a Role (Vendedor ou Gestor).
4. Gestor clica em "Salvar".
5. Frontend dispara `POST /usuarios` contendo o payload.
6. Backend (FastAPI) intercepta a requisição e valida se o requerente tem permissão (`RequireRole`).
7. Backend gera o hash `bcrypt` para a senha padrão corporativa ("senha123").
8. Backend realiza a inserção na tabela `users` (gerando o UUID do usuário).
9. Backend realiza a inserção na tabela `pessoas`, vinculando o `user_id` recém-gerado.
10. Backend retorna `201 Created` e o Frontend atualiza a listagem dinamicamente.

Fluxos Alternativos

A1: O Vendedor Cego (Restrição de Visualização na Listagem)

1. Vendedor acessa a página "Gestão de Pessoas".
2. O botão "+ Novo Funcionário" e os filtros de tipo estão ocultos via CSS/JS (`display: none`).
3. Frontend dispara `GET /pessoas`.
4. Backend identifica pelo token que o usuário é Vendedor.
5. Backend anexa silenciosamente a cláusula `AND user_id IS NULL` na query do Supabase.
6. Frontend renderiza a tabela contendo estritamente Clientes de Balcão.

A2: Escalonamento Vertical Bloqueado (Bypass F12)

1. Um usuário com perfil GESTOR burla o frontend e envia via cURL ou Postman um **POST /usuarios** com **"role": "ADMIN"**.
2. Backend decodifica a requisição e compara a role solicitada com a role de quem pediu.
3. Backend identifica a quebra de hierarquia.
4. Transação é abortada imediatamente com retorno **HTTP 403 Forbidden**.

A3: Alteração de Senha Segura (Meu Painel)

1. Usuário clica em sua Role no header e é direcionado para **meuPainel.html**.
2. Usuário insere "Senha Atual" e "Nova Senha" (2x) e clica em Salvar.
3. Frontend valida se as novas senhas coincidem.
4. Frontend envia **PATCH /auth/senha** sem informar o ID do usuário no body.
5. Backend extrai o **usuario_id** diretamente do **Depends(obter_usuario_atual)**.
6. Backend valida a senha atual e persiste o novo hash no banco.

Regras de Negócio (RN)

ID	Regra	Descrição
RN-01	Dualidade Pessoa-User	Um funcionário obrigatoriamente possui registro em users e pessoas . Um cliente de balcão possui apenas registro em pessoas (user_id IS NULL).
RN-02	Senha Padrão Backend	O frontend não envia senha no cadastro de funcionários. O backend injeta o hash de "senha123" nativamente.
RN-03	Identidade Inviolável	Na rota PATCH /auth/senha , a API rejeita qualquer identificador enviado por Query ou Body, confiando exclusivamente no dono do Token.
RN-04	Inativação (Soft Delete)	Funcionários demitidos ou clientes removidos recebem ativo = false , revogando acesso ao login instantaneamente sem apagar o histórico de vendas.
RN-05	Tolerância a Identificadores	As rotas de leitura e atualização de perfil (/pessoas/{id}) aceitam tanto a chave primária da tabela pessoas quanto o vínculo user_id , resolvendo o mapeamento de forma transparente.

Requisitos Não-Funcionais (RNF)

ID	Atributo	Requisito	Justificativa
RNF-01	Arquitetura	Modularização Frontend	Novas telas isoladas dentro de src/rf-005-gestao-users/frontend/gestaoPessoas e src/rf-005-gestao-users/frontend/meuPainel .
RNF-02	UX	Componentização	Reutilização obrigatória do header.js e global.css do projeto via injeção dinâmica de componentes.

4. PROTÓTIPO FUNCIONAL

Mockup - Tela 1: Gestão de Pessoas (Visão do GESTOR)

☰ ERP Construção		Meu Painel (Gestor)		[Sair]	
👤 Gestão de Entidades		[+ NOVO FUNCIONÁRIO]			
Filtros: Status		[Ativos v]		Tipo [Todos v]	
NOME	CONTATO	PERFIL	STATUS	AÇÕES	

Maria de Fátima	61 9999-9999	Cliente	Ativo	[🗑]	
Carlos (Caixa 1)	carlos@erp.br	Funcionário	Ativo	[🗑]	

Mockup - Tela 2: Meu Pannel (Segurança)

👤	Meu Pannel
Perfil / Role:	[VENDEDOR] (Bloqueado)
E-mail:	[vendedor@erp.com] (Bloqueado)
Nome *:	[Vendedor Teste]
Telefone:	[(61) 98888-7777]
[Alterar Senha] [Salvar Alterações]	

5. ARQUITETURA E ADR

Diagrama de Fluxo (Criação de Usuário e Dualidade de Tabelas)

```

[ Frontend Gestor ]
  |-- POST /usuarios { nome, email, role }
  v
[ FastAPI Backend (RequireRole) ]
  |-- Gera Hash (senha123)
  |
  +--> INSERT INTO users (email, hash, role) -> Retorna UUID (user_id)
  +--> INSERT INTO pessoas (nome, user_id=UUID)

```

ADR-016: Separação de Rotas de Alteração de Senha (Stateful Security)

- **Contexto:** A alteração de senhas precisava ser protegida contra manipulação de requisições, onde um Vendedor poderia tentar interceptar a chamada e enviar o ID de um Gestor no corpo da requisição.
- **Decisão:** A rota `PATCH /auth/senha` foi arquitetada para não aceitar nenhum parâmetro de identificação via Payload ou Query. O identificador do usuário a ser alterado é extraído nativamente no backend através do token de sessão ativo (`Depends(obter_usuario_atual)`).
- **Consequências:** ☒ Mitigação total de IDOR na alteração de senhas. Nenhuma interface frontend pode fraudar a identidade do requisitante.

ADR-017: Controle de Visibilidade Backend (O Vendedor Cego)

- **Contexto:** Por razões de privacidade, Vendedores não devem visualizar a lista de funcionários da empresa, mas precisam gerir clientes de balcão.
- **Decisão:** Em vez de filtrar a lista no frontend (o que exporia os dados na aba Network do navegador), a rota `GET /pessoas` inspeciona a role do token no servidor. Se for `VENDEDOR`, a API modifica a string de busca para o Supabase exigindo `user_id IS NULL`.
- **Consequências:** ☒ Proteção de dados sensíveis na camada de transporte (Backend for Frontend).

ADR-018: Inativação Lógica de Identidades (Soft Delete Integrado)

- **Contexto:** A LGPD exige a possibilidade de exclusão de dados, mas o ERP exige integridade referencial para as Ordens de Serviço faturadas.
- **Decisão:** Executou-se um comando DDL adicionando a coluna `ativo BOOLEAN NOT NULL DEFAULT TRUE` às tabelas `users` e `pessoas`.
- **Consequências:** ☒ Quando um usuário é "excluído" na UI, seu acesso ao sistema (`auth_service`) é imediatamente revogado, mas seu ID permanece atrelado ao histórico contábil.

ADR-019: Resolução Híbrida de Identidade e Tolerância de Chaves

- **Contexto:** Clientes consomem rotas com o ID primário da tabela `pessoas`, enquanto entidades de autenticação referenciam o `user_id`. Uma dessincronização entre o storage do frontend e os endpoints gerava respostas falsas de `404 Not Found`.
- **Decisão:** A camada de serviço (`pessoas_service.py`) foi estruturada com mecanismo de fallback nas operações de busca e atualização: caso a busca primária por `id` não retorne registros, executa-se a busca por `user_id`. O retorno padronizado (`PessoaResponse`) expõe `telefone`, `nome` e os identificadores unificados.
- **Consequências:** ☒ Eliminação de acoplamento rígido de IDs no frontend e prevenção de falhas de navegação cruzada entre microserviços.

6. VALIDAÇÃO DE SEGURANÇA OWASP

O ciclo de consolidação do MVP (v1.0.0) foi homologado por meio de testes práticos de segurança defensiva contra a API local (`http://127.0.0.1:8000`), documentando as respostas reais do servidor Unicorn conforme os critérios de conformidade da rubrica técnica.

Teste 1: A01:2021 – Broken Access Control (Sessão Zumbi / Revogação Imediata via Soft Delete)

Vulnerabilidade: Manutenção de sessões ativas (tokens com validade em cache/tabela) após o desligamento ou inativação do operador, permitindo o uso de sessões zumbis para acesso indevido aos recursos corporativos.

Implementação Defensiva: A validação de sessão ocorre de forma stateful no backend (`obter_usuario_atual`). A cada chamada autenticada, o sistema consulta a sessão e valida a flag `ativo` na tabela de usuários. Caso a conta sofra inativação lógica (`ativo = false`), qualquer sessão ativa associada é recusada em tempo real com `401 Unauthorized`.

Validação Prática:

1. Emissão de credencial ativa para o operador:

```
curl -s -X POST "[http://127.0.0.1:8000/auth](http://127.0.0.1:8000/auth)" \
  -H "Content-Type: application/json" \
  -d '{"email":"vendedor@erp.com", "senha":"senha123"}' | jq
```

```
{
  "role": "VENDEDOR",
  "token": "094ea403-e695-4bb2-97b2-10274efb3ff8",
  "usuario_id": "a4f5d5aa-97a1-45f6-afeb-868fdce15aef",
  "email": "vendedor@erp.com"
}
```

2. Execução da inativação lógica no banco de dados (`UPDATE users SET ativo = false WHERE email = 'vendedor@erp.com';`).
3. Tentativa de consumo de endpoint protegido com o token persistido:

```
curl -i -X GET "[http://127.0.0.1:8000/pessoas](http://127.0.0.1:8000/pessoas)" \
  -H "Authorization: Bearer 094ea403-e695-4bb2-97b2-10274efb3ff8"
```

```
HTTP/1.1 401 Unauthorized
date: Mon, 28 Sep 2026 21:14:59 GMT
server: uvicorn
content-length: 54
content-type: application/json
vary: Origin
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```

```
{"detail": "Token de autenticao ausente ou invalido"}
```

(Comprovação: A invalidação ocorre de forma imediata sem dependência de expiração por tempo [TTL], eliminando janelas de persistência não autorizada).

Teste 2: A04:2021 – Insecure Design (Lógica de Negócio em Troca de Senha)

Vulnerabilidade: Ausência de travas de integridade no fluxo de alteração de credenciais, permitindo reutilização redundante de senhas ou escolha de chaves triviais de baixa entropia.

Implementação Defensiva:

1. **Validação de Schema (Pydantic):** Restrição estrita de comprimento (`min_length=6`), abortando requisições antes do processamento criptográfico.
2. **Regra de Negócio (Service Layer):** Comparação semântica entre `senha_atual` e `nova_senha`, impedindo que o hash seja recalculado para valores idênticos.

Validação Prática:

- **Cenário A: Tentativa de definir nova senha idêntica à atual (400 Bad Request):**

```
curl -i -X PATCH "[http://127.0.0.1:8000/auth/senha]
(http://127.0.0.1:8000/auth/senha)" \
  -H "Authorization: Bearer 950552bd-f516-4f03-a277-c3d58dca800b" \
  -H "Content-Type: application/json" \
  -d '{"senha_atual": "senha123", "nova_senha": "senha123"}'
```

```
HTTP/1.1 400 Bad Request
date: Mon, 28 Sep 2026 21:18:48 GMT
server: uvicorn
content-length: 59
content-type: application/json
vary: Origin
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```

```
{"detail": "A nova senha deve ser diferente da senha atual"}
```

- **Cenário B: Tentativa de envio com comprimento inferior a 6 caracteres (422 Unprocessable Entity):**

```
curl -i -X PATCH "[http://127.0.0.1:8000/auth/senha]
(http://127.0.0.1:8000/auth/senha)" \
  -H "Authorization: Bearer 950552bd-f516-4f03-a277-c3d58dca800b" \
  -H "Content-Type: application/json" \
  -d '{"senha_atual": "senha123", "nova_senha": "123"}'
```

```
HTTP/1.1 422 Unprocessable Entity
date: Mon, 28 Sep 2026 21:19:02 GMT
server: uvicorn
content-length: 154
content-type: application/json
vary: Origin
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block

{"detail":[{"type":"string_too_short","loc":
["body","nova_senha"],"msg":"String should have at least 6
characters","input":"123","ctx":{"min_length":6}}]}
```

Teste 3: A01:2021 – Broken Access Control / BOLA (Segregação de Dados no "Vendedor Cego")

Vulnerabilidade: Exposição indevida de dados internos de colaboradores (IDs, papéis corporativos e e-mails de gestores) para operadores de nível operacional (**VENDEDOR**), caracterizando quebra de autorização a nível de objeto (BOLA).

Implementação Defensiva: O endpoint **GET /pessoas** avalia o perfil contido no token verificado. Se a requisição pertencer a um **VENDEDOR**, o backend injeta forçadamente o filtro **user_id IS NULL** na consulta ao banco, garantindo que registros corporativos sequer trafeguem pela rede.

Validação Prática:

```
curl -i -X GET "[http://127.0.0.1:8000/pessoas]
(http://127.0.0.1:8000/pessoas)" \
  -H "Authorization: Bearer 950552bd-f516-4f03-a277-c3d58dca800b"
```

```
HTTP/1.1 200 OK
date: Mon, 28 Sep 2026 21:19:13 GMT
server: uvicorn
content-length: 822
```

```
content-type: application/json
vary: Origin
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```

```
[
  {"id": "175e327c-1969-4bf1-98e2-d04b97881eeb", "nome": "Cliente Balcão Silva", "cpf": "99988877766", "telefone": null, "user_id": null, "ativo": true},
  {"id": "44b46834-792c-4efb-8cbe-82e78733c687", "nome": "CLIENTE TESTE", "cpf": "12345678999", "telefone": null, "user_id": null, "ativo": true},
  {"id": "5098c816-4d46-46ca-9898-7130e912e6cd", "nome": "janaina brandao", "cpf": "03450071799", "telefone": null, "user_id": null, "ativo": true},
  {"id": "6eb46104-5e45-433a-891c-2204e5975afc", "nome": "João pedro silva de lima", "cpf": "03945007178", "telefone": null, "user_id": null, "ativo": true},
  {"id": "bc395d0b-46b4-4c7c-8526-1dab4b7ec68f", "nome": "Lucas the nego cluadio", "cpf": "84567897465", "telefone": null, "user_id": null, "ativo": true},
  {"id": "c2af72f0-d806-4c16-9d47-5b5a0cdabc5af", "nome": "Teste", "cpf": "09876543211", "telefone": null, "user_id": null, "ativo": true}
]
```

(Comprovação: 100% dos registros devolvidos possuem "user_id": null. Nenhuma informação sobre a equipe de colaboradores ou credenciais corporativas foi exposta).

Teste Adicional: A01:2021 – Broken Access Control (Escalonamento Vertical via Criação)

Vulnerabilidade: Um operador com perfil **GESTOR** intercepta a chamada **POST /usuarios** e altera o payload para **"role": "ADMIN"**, tentando autopromover-se a superusuário.

Implementação Defensiva: Validação cruzada estrita entre o token do requisitante e o payload submetido, com bloqueio imediato caso a role solicitada seja igual ou superior à do solicitante.

Validação Prática:

```
curl -i -X POST "[http://127.0.0.1:8000/usuarios]
(http://127.0.0.1:8000/usuarios)" \
-H "Authorization: Bearer ee1763ec-e9fa-4e5d-8cd9-ac774fd4760f" \
-H "Content-Type: application/json" \
-d '{"nome": "Conta Hacker", "email": "hacker@erp.com", "role": "ADMIN"}'
```

```
HTTP/1.1 403 Forbidden
date: Mon, 28 Sep 2026 21:06:22 GMT
server: uvicorn
content-length: 51
```

```
content-type: application/json
vary: Origin
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block

{"detail": "GESTOR não pode criar usuário ADMIN."}
```